



École Supérieure de Technologie de Nador Filière ILCS – S3

COMPTE RENDU DE TRAVAUX PRATIQUES

**TP N°2 : Prise en main de l'environnement, Anatomie d'un projet &
Débogage LogCat Module Développement Mobile**

Réalisé par: SARA HAJJI

HANANE EL AASRAOUI

Encadré par :Ouzayr RABHI

Année universitaire 2026/2027

1. Objectifs du TP 1

- Lancer Android Studio et créer un premier projet Android vierge. Configurer et exécuter l'application sur un émulateur AVD ou un smartphone physique via ADB.
 - Explorer et modifier l'arborescence du projet (AndroidManifest.xml, java/, res/).
- Maîtriser LogCat (Log.i, Log.d, Log.e) et le débogage par points d'arrêt.

2. Environnement de travail

Élément	Valeur
IDE	Android Studio
Langage	Java
Nom du projet	MonPremierProjet
Package	com.estn.monpremierprojet
Minimum SDK	API 21 : Android 5.0 (Lollipop)
Modèle	Empty Views
Activity	Cible d'exécution [Smartphone physique via ADB / Émulateur AVD]

3. Partie 1 : Prise en main & Hello World

3.1 Création du projet :

Le projet A été créé via New Project, avec le modèle Empty Views Activity et les paramètres du tableau ci-dessus. Après la fin de la synchronisation de Gradle, la structure du projet est apparue dans le volet de gauche.

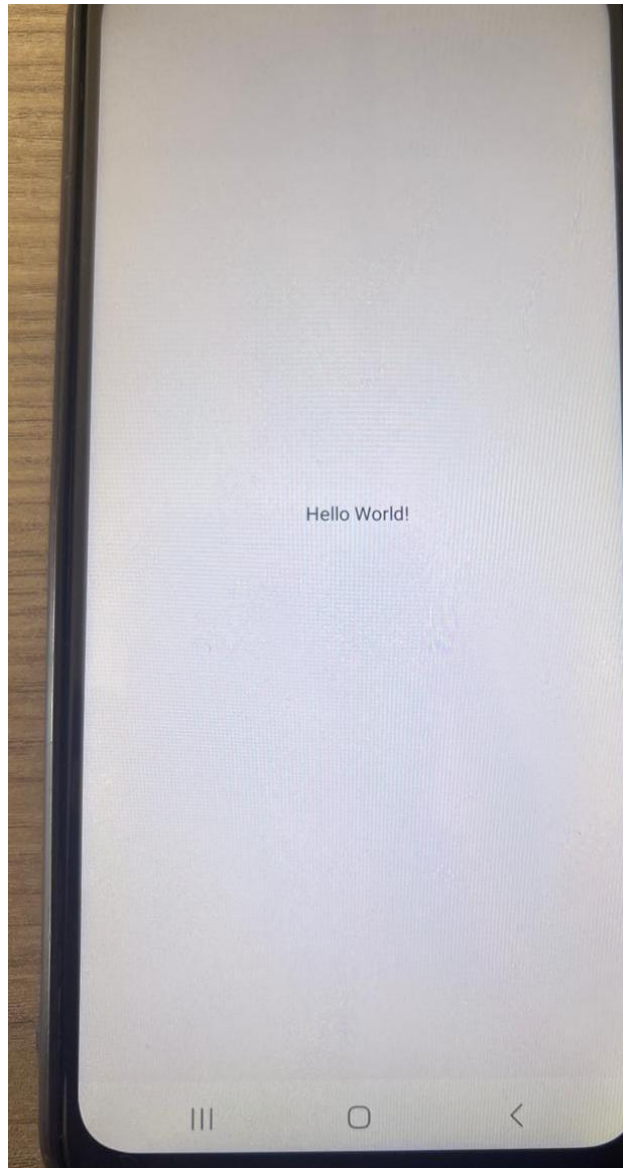
3.2 Configuration de la cible d'exécution:

Deux options sont possibles :

- Smartphone physique : activation des Options pour développeurs (7 appuis sur le Numéro de build), activation du Débogage USB, connexion par câble USB puis autorisation du débogage sur le téléphone.
- Émulateur AVD : Tools > Device Manager > Create Virtual Device, choix d'un profil (ex. Pixel 6) et téléchargement de l'image système. Remarque : lors du téléchargement de l'image système (16 KB Page Size, API 37.2), l'erreur « Connection reset » est apparue à cause d'une coupure de connexion. La solution consiste à choisir une image plus légère (ex. API 34 ou 35, Google APIs x86_64), à vérifier la connexion Internet, ou à utiliser un smartphone physique.

3.3 Exécution de l'application :

L'application « Hello World » a été lancée avec le bouton Run



Question : Quel est le fichier généré à la fin de la compilation pour être installé sur le téléphone ?

Réponse : le fichier généré est un fichier APK (Android Package), par exemple appdebug.apk. Il regroupe le code compilé, les ressources et le manifeste, et c'est lui qui est installé sur l'appareil.

4. Partie 2 : Anatomie du projet & Débogage

4.1 Modification de l'arborescence res/:

- Dans res s/values/strings.xml, deux chaînes ont été ajoutées :

```
<string name="welcome_message">Bienvenue dans le TP2 de  
Développement Mobile !
```

```
</string>
```

```
<string name="btn_click">Cliquer ici</string>
```

- Dans res/values/colors.xml, une nouvelle couleur a été définie:

```
<color name="primary_estn">#0056B3</color>
```

- Dans res/layout/activity_main.xml, le TextView utilise désormais @string/welcome_message et un Button a été ajouté en dessous :

```
<Button
```

```
    android:id="@+id/btnTest"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="@string/btn_click" />
```

4.2 Débogage avec LogCat

- Dans MainActivity.java, la classe Log a été importée, une constante TAG a été déclarée et des logs de différents niveaux ont été ajoutés dans onCreate() :

```
import android.util.Log;
private static final String TAG = "TP2_DEBUG";
// dans onCreate()
Log.i(TAG, "L'activité MainActivity a été créée avec succès."); // Info
Log.d(TAG, "Test de valeur de débogage : API Level = " +
android.os.Build.VERSION.SDK_INT); // Debug
Button btn = findViewById(R.id.btnTest);
btn.setOnClickListener(v -> {
Log.e(TAG, "Le bouton a été cliqué par l'utilisateur !"); // Error
});
```

- Dans la fenêtre Logcat, le filtre tag:TP2_DEBUG a été utilisé, puis les niveaux de sévérité (Info, Debug, Error) ont été testés.

[Capture d'écran : Logcat filtré par le TAG TP2_DEBUG]

4.3 Points d'arrêt (Breakpoints)

5. Un point d'arrêt (point rouge) a été placé dans la marge, à gauche de la ligne Log.e(TAG, ...).

6. L'application a été lancée en mode Debug (icône / Shift + F9). 

7. Un clic sur le bouton a arrêté l'exécution sur le point d'arrêt.

8. Le volet Variables a été inspecté et Step Over (F8) a permis d'avancer ligne par ligne.

Rôle du fichier AndroidManifest.xml

AndroidManifest.xml est le fichier de description de l'application. Il fournit au système

Android les informations essentielles : le nom du package, les composants de l'application

(Activities, Services, etc.), l'activité principale lancée au démarrage (intent-filter MAIN /

LAUNCHER), les permissions demandées, ainsi que l'icône et le nom de l'application. Sans ce fichier, le système ne peut pas exécuter l'application.

Pourquoi utiliser strings.xml plutôt que du texte « en dur » ?

Internationalisation : on peut ajouter facilement d'autres langues (values-fr, values-ar, etc.) sans toucher au code.

Maintenance : un texte utilisé à plusieurs endroits se modifie en un seul endroit.

Séparation du contenu et de la présentation : le code et les layouts restent propres et lisibles.

Bonne pratique Android : Android Studio signale les textes hardcodés comme avertissement.

6. Conclusion

Ce TP a permis de créer et d'exécuter un premier projet Android, de comprendre l'organisation d'un projet (manifeste, code Java, ressources), et de maîtriser les outils de débogage : LogCat pour tracer l'exécution et les points d'arrêt pour inspecter les variables pas à pas. Ces compétences sont indispensables pour les prochains TP de développement mobile.

